

Advanced Software Engineering

Comprehensive Self-Study Quiz Deck (Chapters 1–3)

"Tell me and I forget, teach me and I may remember, involve me and I learn." — Benjamin Franklin

”

 3 Core Chapters

 31 Conceptual Questions

 Instant Answer Feedback

 Self-Paced Learning

Self-Study Knowledge Map & Chapter Navigation

Select any chapter below to begin self-testing, or use the header dropdown above to jump to any specific question.

”

Chapter 1

Introduction to SE

Software crisis, IEEE definition, core activities, Brooks's law, SoC, ISO 25010, ethics & AI churn.

- **Questions:** Q01 – Q09 (9 Qs)
- **Target:** Core Mental Models

[Start Chapter 1 →](#)

Chapter 2

Process Models

Waterfall, V-Model, MVP, Agile Manifesto, Kanban WIP, Scrum, Tech Debt, XP, TDD & CI/CD.

- **Questions:** Q10 – Q23 (14 Qs)
- **Target:** Workflow & Agility

[Start Chapter 2 →](#)

Chapter 3

Requirements Eng.

User vs. System, Domain, Measurable NFRs, 5 Whys, Tacit Knowledge, Use Cases & AI risks.

- **Questions:** Q24 – Q31 (8 Qs)
- **Target:** Elicitation & Specs

[Start Chapter 3 →](#)

Chapter 1. Introduction to Software Engineering

Foundations, Complexity, Professional Ethics & AI Impact

"The first false assumption: All programmers are created equal. The second: More programmers will make it faster." — Fred Brooks

”

 9 Questions (Q01 – Q09)

Ch 1. Introduction • Question 01

Topic: Software Crisis & Complexity

Why couldn't the 1968 Software Crisis be resolved simply by purchasing faster computer hardware or larger memory?

- ☐ A Computer hardware manufacturing and memory fabrication completely stagnated in the late 1960s, preventing computational speedups.
- ☒ B The crisis was fundamentally an intellectual and organizational challenge of system complexity, which faster hardware only amplified.
- ☐ C Programming languages of that era strictly lacked mathematical calculation primitives and compiler memory allocation capabilities.
- ☐ D Early mainframe computers were physically incompatible with shared telecommunication networks and multi-terminal architectures.

Ch 1. Introduction • Question 02

Topic: IEEE Software Definition

According to the IEEE standard definition of software, which of the following is NOT considered a component of software?

- ☐ A Executable computer programs and source code files.
- ☐ B System database schemas and configuration files.
- ☒ C CPU processor hardware and physical memory units.
- ☐ D Software installation and deployment procedures.

Ch 1. Introduction • Question 03

Topic: Core Engineering Activities

Which of the following pairs correctly matches a specific software engineering action with its corresponding universal core activity?

- A** Conducting stakeholder interviews to draft user stories $\rightarrow$ Software Specification
- B** Writing automated unit tests to mock database responses $\rightarrow$ Software Design & Implementation
- C** Refactoring database schemas to improve query speed $\rightarrow$ Software Validation
- D** Swapping a third-party payment API for a new gateway $\rightarrow$ Software Specification

Ch 1. Introduction • Question 04

Topic: Brooks's Law & Team Scaling

A project is 3 weeks behind schedule with 2 weeks remaining before release. The manager hires 4 junior programmers to speed up progress. What will happen according to Brooks's Law?

- ☐ A The project will finish 1 week early.
- ☒ B The project will be delayed further because senior engineers must spend time onboarding and mentoring new hires.
- ☐ C The existing developers will code twice as fast.
- ☐ D Communication complexity remains unchanged.

Ch 1. Introduction • Question 05

Topic: Separation of Concerns

An order-processing module directly handles HTTP requests, executes payment transactions, queries the SQL database, and generates HTML receipt emails. Which fundamental design principle is most severely violated?

- A** Separation of Concerns: Multiple distinct responsibilities are tightly tangled in a single module.
- B** YAGNI: Speculative future features are implemented before actual business requirements emerge.
- C** Brooks's Law: Adding developers to the order module increases communication complexity exponentially.
- D** Anticipation of Change: System configurations are hardcoded into compiled production binaries.

Ch 1. Introduction • Question 06

Topic: ISO 25010 Quality Model

Which of the following matches a real-world software issue with its corresponding ISO 25010 quality characteristic?

- A A database query taking 15 seconds to return results $\rightarrow$ Maintainability (Testability)
- B A system crash occurring when a third-party API goes offline $\rightarrow$ Reliability (Fault Tolerance)**
- C Developers struggling to write unit tests due to tight coupling $\rightarrow$ Portability (Adaptability)
- D An unencrypted session cookie allowing account takeover $\rightarrow$ Usability (Operability)

Ch 1. Introduction • Question 07

Topic: ACM/IEEE Code of Ethics

Under the ACM/IEEE Code of Ethics, if an employer directs an engineer to implement an algorithm that falsifies safety compliance reports, what is the engineer's obligation?

- ☐ A Comply, because the employer pays the engineer's salary.
- ☒ B Refuse and escalate, because the Public Interest takes precedence over Employer loyalty.
- ☐ C Implement the code but omit documentation.
- ☐ D Outsource the code to an external vendor.

Ch 1. Introduction • Question 08

Topic: Code Churn & AI Assistants

In empirical studies evaluating AI coding assistants (such as GitClear's analysis of 150M lines of code), "Code Churn" emerged as a major warning sign. What does high Code Churn indicate in an AI-assisted codebase?

- A** Code is rapidly rewritten, deleted, or patched shortly after commit, indicating brittle code accepted without sufficient verification.
- B** Compilers and bundlers are aggressively removing unreachable dead code from application binaries during automated deployment.
- C** Software engineering teams are switching programming languages frequently due to automated polyglot syntax translation.
- D** Automated test cases are executing too quickly and depleting available CI/CD pipeline virtual machine compute resources.

Ch 1. Introduction • Question 09

Topic: GenAI Architectural Slop Risk

An engineer prompts an AI to generate a complex payment calculation module, and then asks the same AI to write unit tests without providing a formal specification. All tests pass. What is the primary risk?

- A** Echo-chamber validation: The generated tests merely mirror the AI's internal flawed assumptions rather than actual business requirements.
- B** Performance bottleneck: AI-generated test assertions take significantly longer to execute than human-written assertions.
- C** Compilation failure: Testing frameworks cannot parse automated mock datasets generated by large language models.
- D** Version lock-in: The test suite becomes tightly coupled to a single specific cloud runtime environment.

Chapter 2. Software Development Processes & Methodologies

Plan-Driven vs. Agile, Scrum, Kanban, Technical Debt & Extreme Programming

"Individuals and interactions over processes and tools; working software over comprehensive documentation." — Agile Manifesto (2001)

”

 14 Questions (Q10 – Q23)

Ch 2. Development Processes • Question 10

Topic: Waterfall Late Integration Risk

What is the primary operational drawback of the traditional Waterfall model?

- ☐ A It produces inadequate documentation for external auditing and compliance.
- ☒ B It makes accommodating changing requirements extremely difficult and costly once underway.
- ☐ C It eliminates the need for component and system testing during execution.
- ☐ D It cannot be deployed across large-scale multi-site engineering organizations.

Ch 2. Development Processes • Question 11

Topic: V-Model Verification & Validation

What is the primary engineering advantage of the V-Model over the classic Waterfall model?

- ☐ A It produces working software increments in short two-week sprint iterations.
- ☒ B It enforces test planning and acceptance criteria design concurrently with early specification phases.
- ☐ C It eliminates the need for detailed architecture design and interface contracts.
- ☐ D It allows customers to dynamically modify requirements at zero cost during implementation.

Ch 2. Development Processes • Question 12

In software process engineering, what is the fundamental conceptual difference between "Incremental" and "Iterative" development?

- A** Incremental focuses on automated testing; Iterative focuses on UI design.
- B** Incremental delivers finished functional slices stage-by-stage; Iterative refines an end-to-end working draft over repeated cycles.
- C** Incremental is managed by product owners; Iterative is managed exclusively by external regulators.
- D** Incremental follows waterfall rules; Iterative produces no documentation.

Topic: Incremental vs. Iterative Process

Ch 2. Development Processes • Question 13

In Henrik Kniberg's famous Minimum Viable Product (MVP) analogy (Skateboard to Car), why is delivering a standalone car wheel in the first release considered an anti-pattern?

- ☐ A Because manufacturing an isolated wheel is significantly more expensive than building a skateboard.
- ☒ B Because an isolated wheel provides zero end-to-end transportation value, preventing users from validating core problem assumptions.
- ☐ C Because modern vehicle designs prohibit upgrading wheels into scooters.
- ☐ D Because software engineering standards mandate that all early increments must be rectangular.

Topic: Kniberg's MVP Analogy (Skateboard)

Ch 2. Development Processes • Question 14

Topic: Agile Manifesto Living Software

The Agile Manifesto states: *"Working software over comprehensive documentation."* What does this value primarily advocate in practice?

- ☐ A Engineering teams are completely prohibited from writing architecture blueprints or API specifications.
- ☒ B While documentation has value, delivering working, tested, and validated software is the primary measure of progress and customer value.
- ☐ C Projects should be evaluated solely on executive PowerPoint presentations.
- ☐ D Source code comments must be erased before deployment to production.

Ch 2. Development Processes • Question 15

Topic: Agile Sustainable Development Pace

Agile Principle 8 states: *"Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely."***** What is the primary engineering motivation?

- A** To prevent code quality degradation, accumulated defects, and developer burnout caused by chronic overtime and crunch periods.
- B** To mandate that developers submit at least 50 pull requests per day.
- C** To eliminate the need for software upgrades after initial system launch.
- D** To restrict development teams to working only on legacy systems.

Ch 2. Development Processes • Question 16

Topic: Kanban WIP Limits & Flow Control

In the Kanban process framework, what is the primary operational purpose of enforcing strict "Work In Progress" (WIP) limits on columns?

- ☐ A To prevent developers from modifying automated unit test scripts.
- ☒ B To expose bottlenecks, reduce multitasking context-switching waste, and maximize delivery flow throughput.
- ☐ C To mandate that every team member attends daily 15-minute standup meetings.
- ☐ D To ensure all software increments are packaged into fixed 2-week sprint iterations.

Ch 2. Development Processes • Question 17

In the Scrum framework, what is the primary operational objective of the **Sprint Retrospective**** held at the end of each sprint?**

- A** To demonstrate working software increments to external business stakeholders.
- B** To inspect internal team collaboration, engineering practices, and tools, and identify actionable process improvements for the next sprint.
- C** To assign individual performance ratings and conduct annual salary reviews.
- D** To rewrite the entire product backlog and discard unfinished user stories.

Topic: Daily Scrum Coordination Goals

Ch 2. Development Processes • Question 18

Topic: Technical Debt Metaphor

According to Ward Cunningham's Technical Debt metaphor, what represents the "compounding interest" paid by a software organization?

- ☐ A The annual licensing fees paid for cloud hosting infrastructure and IDEs.
- ☒ B The ongoing extra time, degraded velocity, and regression defects suffered during all future development.
- ☐ C The bonus compensation paid to engineers who complete sprint tickets ahead of schedule.
- ☐ D The legal costs of acquiring open-source third-party dependencies.

Ch 2. Development Processes • Question 19

Martin Fowler coined the term **"Flaccid Scrum" to describe which critical software engineering failure?**

- A** Adopting Scrum management ceremonies (daily standups, sprints, story points) while completely neglecting technical engineering practices like TDD, refactoring, and CI.
- B** Refusing to use Jira or commercial issue tracking software in favor of physical sticky notes.
- C** Allowing product owners to adjust backlog priorities between sprint planning sessions.
- D** Enforcing automated test execution on every Git commit in the continuous integration pipeline.

Topic: Martin Fowler's Flaccid Scrum

Ch 2. Development Processes • Question 20

In Extreme Programming (XP), what is the primary role of the "Navigator" during a pair programming session?

- ☐ A Typing out code syntax and executing local terminal commands.
- ☒ B Thinking strategically, reviewing code in real time, considering edge cases, and looking at the broader architecture.
- ☐ C Serving as the official sprint facilitator and managing Jira backlog ticket status.
- ☐ D Negotiating customer contracts and approving annual engineering budgets.

Topic: XP Pair Programming Roles

Ch 2. Development Processes • Question 21

Topic: TDD Red-Green-Refactor Cycle

In Extreme Programming's Test-Driven Development (TDD), what is the specific objective of the `Refactor` step in the Red-Green-Refactor cycle?

- A** Adding new functional capabilities and expanding the module's public API contract.
- B** Improving the internal structure and readability of the code while ensuring all existing automated tests continue to pass.
- C** Removing unit tests that take longer than one second to execute in the local test suite.
- D** Rewriting the application from an object-oriented language to a functional programming language.

Ch 2. Development Processes • Question 22

What is the defining operational distinction between "Continuous Delivery" and "Continuous Deployment"?

- A** Continuous Delivery requires manual code compilation; Continuous Deployment automates compilation.
- B** Continuous Delivery stops at staging and requires human business approval to release; Continuous Deployment automatically deploys passing builds directly to live production.
- C** Continuous Delivery is used solely for mobile apps; Continuous Deployment is used solely for backend databases.
- D** Continuous Delivery eliminates unit testing; Continuous Deployment mandates pair programming.

Topic: Continuous Delivery vs. Deployment

Ch 2. Development Processes • Question 23

In modern AI Specification-Driven development, what is the primary role of automated CI/CD verification gates?

- A** To prevent human developers from reviewing artificial intelligence output.
- B** To enforce deterministic quality, test compliance, and defect containment before AI-generated code merges into production.
- C** To convert natural language prompts directly into cloud infrastructure invoices.
- D** To replace software specifications with unverified prompt histories.

Topic: Automated CI/CD Quality Gates

Chapter 3. Requirements Engineering

Elicitation, Functional & Non-Functional Specifications, Modeling & Validation

"The hardest single part of building a software system is deciding precisely what to build." — Fred Brooks (1987)

”

 8 Questions (Q24 – Q31)

Ch 3. Requirements Engineering • Question 24

Topic: User vs. System Requirements

In requirements engineering, what is the critical operational distinction between **User Requirements** and **System Requirements**?

- A** User requirements are high-level stakeholder goals; system requirements are detailed functional contracts for developers.
- B** User requirements specify UI wireframes; system requirements specify backend database schemas.
- C** User requirements can never change; system requirements are refactored continuously during daily scrums.
- D** User requirements come from external legal auditors; system requirements are generated by compiler tools.

Ch 3. Requirements Engineering • Question 25

Topic: Domain Requirements (UberEats)

Consider the following specification for the UberEats platform:
“Due to municipal food hygiene regulations, perishable warm food delivery transit time shall not exceed 45 minutes, and containers must maintain a temperature above 60°C throughout transit.”
What category of software requirement does this statement represent?

- A** Domain Requirement (a constraint imposed by the operational environment, industry regulations, or physical laws)
- B** Functional Requirement (a specification of an active software computation, user feature, or system service)
- C** Non-Functional Requirement (a general software quality attribute concerning performance, scalability, or uptime)
- D** User Requirement (a high-level, natural-language goal or business vision expressed by end consumers)

Ch 3. Requirements Engineering • Question 26

Topic: Verifiable & Measurable NFRs

A client provides an imprecise non-functional goal: **"The order checkout system must be blazing fast and highly reliable."** Which of the following correctly transforms this vague goal into a ****verifiable, testable engineering metric****?

- A** 99% of checkouts shall have response time ≤ 500 ms, and peak uptime shall be $\geq 99.95\%$.
- B** The checkout UI shall use sleek animations so customers perceive maximum speed.
- C** All checkout services shall use memory-safe code to guarantee bug-free execution.
- D** The cloud database shall allocate unlimited RAM whenever transaction load increases.

Ch 3. Requirements Engineering • Question 27

Topic: Root Cause & The 5 Whys Heuristic

During a requirements elicitation interview, an executive insists: *"Our clinical software must include a blockchain ledger to record patient vitals."* What is the most effective engineering interview heuristic to apply?

- A** Apply the "5 Whys" to investigate the underlying data integrity and audit problem rather than prematurely locking in the suggested technology.
- B** Immediately begin drafting smart contracts and relational database schemas for the requested blockchain feature.
- C** Reject the executive's request outright because non-technical stakeholders are prohibited from proposing system capabilities.
- D** Politely terminate the interview and switch exclusively to passive workplace observation.

Ch 3. Requirements Engineering • Question 28

Topic: Ethnography & Tacit Knowledge

In requirements engineering, why is **Ethnography (workplace observation)** uniquely vital when analyzing complex operational environments such as hospital emergency rooms or air traffic control?

- A** It uncovers tacit knowledge—ingrained habits, physical workarounds, and unwritten shortcuts that users perform automatically but never mention during interviews.
- B** It automatically compiles natural language requirements directly into executable acceptance test suites without human intervention.
- C** It eliminates the need for subsequent software architecture design, database modeling, or code review phases.
- D** It guarantees that the resulting software requirements will achieve 100% mathematical completeness on the initial development sprint.

Ch 3. Requirements Engineering • Question 29

Topic: Use Case Diagrams vs. Descriptions

A software engineering team creates a UML Use Case Diagram showing an actor connected to the "Withdraw Cash" use case. Why must engineers author a detailed textual **Use Case Description**** in addition to the diagram?**

- A** Diagrams only show high-level scope; descriptions define sequential flows, preconditions, postconditions, and exception handling.
- B** UML diagrams cannot be rendered by web browsers without accompanying markdown text.
- C** Compilers require use case descriptions to allocate heap memory for actor threads.
- D** Descriptions convert non-functional requirements into automated GUI wireframes.

Ch 3. Requirements Engineering • Question 30

Why is fixing a requirements error after software delivery significantly more expensive than fixing an error during early development?

- ☐ A Requirements documents cannot be legally modified once signed by clients.
- ☒ B A late fix requires redesigning, recoding, retesting, and redeploying cascading components that were built on the flawed premise.
- ☐ C Compilers automatically lock code repositories against changes after the first production release.
- ☐ D Automated unit tests lose their validity after code has been deployed to cloud environments.

Topic: Exponential Cost of Late Defects

Ch 3. Requirements Engineering • Question 31

Topic: LLM Hallucinations in Requirements

When software engineering teams use Large Language Models (LLMs) to generate requirements specifications from stakeholder interview transcripts, what is the primary operational risk requiring human-in-the-loop verification?

- A** The LLM may hallucinate plausible-sounding but fictitious business logic, omitted edge-case constraints, and non-existent external API integrations.
- B** The LLM cannot output text formatted in Markdown bullet points or standard user story Given-When-Then acceptance criteria.
- C** The LLM will consume excessive server memory and cause database deadlock errors across production microservice clusters.
- D** The LLM strictly enforces waterfall development practices and refuses to generate requirements for iterative agile sprint cycles.



Congratulations! Quiz Deck Completed

You Have Reviewed All 31 Conceptual Milestones

"Continuous improvement is better than delayed perfection." — Mark Twain „



Review Ch 1

Revisit Software Crisis, ISO 25010 & Ethics concepts.

[Jump to Q01 →](#)



Review Ch 2

Revisit Agile, Scrum, Kanban & TDD cycles.

[Jump to Q10 →](#)



Review Ch 3

Revisit Elicitation, NFR metrics & Use Cases.

[Jump to Q24 →](#)